

ECMWF Software with Documentation (SD) Deliverable

Project / Activity Name: GRUAN Radiosonde Data Processing System

Document Version: 1.0.1 **Date:** 2025-12-03 **Last Updated:** 2026-02-25

Author(s): Emanuele Tramutola - CNR, C.I.A.O. Research Group, IMAA Institute, Tito Scalo (PZ), Italy

1. Purpose

The purpose of this software is to provide a high-performance, parallel data processing system for importing and managing GRUAN (GCOS Reference Upper-Air Network) radiosonde data from multiple instruments (Vaisala RS41, RS92 and Meisei RS11G, IMS-100) into a PostgreSQL database. It ensures that scientific data is efficiently extracted, transformed, and stored for analysis.

2. Audience

External (public) audience. The software and its documentation are intended for meteorologists, climate researchers, and data managers who need to process and analyze GRUAN radiosonde data.

3. Software Repository

Repository URL: https://github.com/emanueletramutola/gruan_retriever

Clone URL:

HTTPS

```
git clone https://github.com/emanueletramutola/gruan_retriever.git
```

SSH

```
git clone git@github.com:emanueletramutola/gruan_retriever.git
```

Repository Structure:

- **config/** - Configuration files
- **converters/** - Data transformation logic
- **database/** - Database operations and migration scripts
- **processors/** - Core data processing logic

- `readers/` - NetCDF file readers
- `tests/` - Unit and integration tests
- `utils/` - Utility functions (logging, station management)
- `main.py` - Main entry point
- `requirements.txt` - Python dependencies
- `README.md` - Quick start guide
- `SoftwareUserGuide.md` - Detailed user manual

License: CC-BY-4.0

4. Technical Specifications

Programming Language: Python 3.12+ (minimum required version)

Package Manager / Environment: pip / venv

Supported Operating Systems:

- Linux (tested on Ubuntu/Debian)
- macOS
- Windows

System Requirements:

- **CPU:** Multi-core processor recommended for parallel processing
- **RAM:** 8GB minimum recommended
- **Storage:** Dependent on data volume (PostgreSQL database size)
- **Database:** PostgreSQL 12+

Software Dependencies:

Dependency	Purpose
<code>beautifulsoup4</code>	Web scraping for station data
<code>requests</code>	HTTP requests for station data
<code>numpy</code>	Numerical data processing
<code>pandas</code>	Data manipulation and transformation
<code>netCDF4</code>	Reading NetCDF data files
<code>sqlalchemy</code>	Database ORM and connection management
<code>psycopg2</code>	PostgreSQL adapter
<code>pyyaml</code>	Configuration file parsing
<code>psutil</code>	System resource monitoring
<code>metpy</code>	Atmospheric thermodynamics calculations
<code>scipy</code>	Signal processing (smoothing)

External Data Requirements:

- GRUAN data files (NetCDF format, typically packaged in JAR/ZIP archives)
 - Access to GRUAN website for station metadata (optional, for updates)
-

5. Installation Instructions

Prerequisites

- **No system administrator privileges required** (for Python setup)
- Git installed (optional, for cloning repository)
- **Python 3.12 or higher** (minimum required version)
- PostgreSQL 12+ database server (requires admin privileges for initial setup)

Installation Steps

1. Clone the repository:

```
git clone https://github.com/emanueletramutola/gruan_retriever.git
cd gruan_retriever
```

2. Create and activate a virtual environment:

```
python -m venv .venv
source .venv/bin/activate # Linux/Mac
# OR
.venv\Scripts\activate   # Windows
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. **Configure Environment Variables:** Set the database password environment variable:

```
export GRUAN_USER_PSW="your_database_password"
```

5. Verify installation:

```
python -c "import processors; print('Installation successful')"
```

Troubleshooting

Issue: ModuleNotFoundError **Solution:** Ensure the virtual environment is activated (`source .venv/bin/activate`).

Issue: Database connection failed **Solution:** Verify `config.yaml` settings and ensure `GRUAN_USER_PSW` is set. Check if PostgreSQL is running.

6. Quick Start Guide

For users who want to get started immediately:

```
# 1. Setup environment
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export GRUAN_USER_PSW="your_password"

# 2. Configure paths
# Edit config.yaml to point to your data directories

# 3. Run the processor
python main.py
```

Expected output: Logs indicating the start of the import process, progress updates for each file, and a final execution summary.

7. Usage Instructions

Input/Output Formats

Supported Input Formats: - GRUAN Data Products (NetCDF files inside JAR/ZIP archives) - Supported Sondes: - **RS92:** Vaisala RS92 radiosonde - **RS41:** Vaisala RS41 radiosonde - **RS11G:** MEISEI RS-11G radiosonde - **IMS100:** MEISEI IMS-100 radiosonde

Output Formats: - PostgreSQL Database Tables (configurable table names, default: `header` and `data` tables for each sonde type) - Note: Table names can be customized in `config.yaml` under `database.table_names`

Configuration

Configuration is managed via `config.yaml`. Key sections:

Parameter	Description
<code>paths</code>	Directory paths for each sonde type (e.g., <code>rs92</code> , <code>rs41</code> , <code>ims100</code> , <code>rs11g</code>)
<code>database</code>	Connection details (host, port, user, dbname)
<code>logging</code>	Log levels and file paths

Example `config.yaml` snippet:

```
paths:
  rs92: "/path/to/rs92/data"
  rs41: "/path/to/rs41/data"
  ims100: "/path/to/ims100/data"
  rs11g: "/path/to/rs11g/data"

database:
  configuration:
    dbname: "gruan"
    user: "gruan_user"
    host: "localhost"
```

Command-Line Interface

Basic usage:

```
python main.py
```

The script automatically processes all configured sonde types in parallel.

Use Cases

Use Case 1: Full Data Import

Description: Import all available data for all configured sonde types.

Command:

```
python main.py
```

Expected output: The system iterates through RS92, RS41, etc., processes files in parallel, and updates the database.

Use Case 2: Update Station Database

Description: Fetch latest station metadata from GRUAN website.

Command:

```
# Create a script or use python shell
from utils.station_manager import update_station_database
update_station_database()
```

8. Atmospheric Diagnostics Module

The software includes a dedicated module for computing a wide range of thermodynamic and stability indices from radiosonde profiles. This module can be used independently after data retrieval (e.g., from the database or directly from NetCDF files) to derive quantities such as CAPE, CIN, LCL, LFC, EL, precipitable water, wind shear, and cloud base heights.

Dependencies

The module relies on: - `numpy` – numerical operations - `scipy.ndimage.uniform_filter1d` – smoothing - `metpy` – unit-aware atmospheric calculations

These libraries are included in `requirements.txt`.

Module Overview

The module is designed to be imported and used in a Python script or interactive session. All inputs must be provided as **MetPy Quantities** with appropriate units.

Utility Functions

- `smooth(x, n=5)` – applies a uniform moving average filter to array `x` (size `n`) with 'nearest' boundary handling.
- `ensure_monotonic(p, *args)` – sorts all arrays in descending order of pressure `p` (high to low), ensuring monotonic profiles.

Core Thermodynamics

- `compute_basic_thermo(p, T, RH)` – returns dewpoint (`Td`), potential temperature (`theta`), and equivalent potential temperature (`theta_e`).

Convection & Parcel Diagnostics

- `compute_parcel_diagnostics(p, T, Td)` – computes the parcel profile, LCL, LFC, EL, CAPE, and CIN using the surface parcel (first level).

Instability Indices

- `compute_indices(p, T, Td)` – returns Lifted Index, K-Index, Total Totals Index, and Showalter Index.

Tropopause

- `compute_tropopause(p, T)` – estimates the tropopause pressure and temperature.

Wind Diagnostics

- `compute_wind(p, z, u, v)` – calculates bulk wind shear over 0-1 km, 0-3 km, 0-6 km and storm-relative helicity (SRH) in the 0-3 km layer.

Other Diagnostics

- `compute_misc(p, T, Td)` – precipitable water (PWAT) and the index of the first freezing level.

Cloud Base Height

Two methods are provided: - `cloud_base_low(z, RH, zmax=3000*m, rh_crit=0.95)` – finds the lowest level below `zmax` where relative humidity exceeds `rh_crit`. - `cloud_base_zhang(z, RH, T, zmin=4000*m, rh_min=0.75)` – applies a smoothed RH profile and detects local maxima above `zmin` (based on Zhang et al. 2010).

The function `cloud_base_height(z, RH, T)` combines both, returning the cloud base height and a description of the method used.

Master Function

- `full_sounding_diagnostics(p, T, RH, u, v, z)` – orchestrates all calculations and returns a comprehensive dictionary with the following keys:
 - "Thermo" – Td, Theta, Theta_e
 - "Parcel" – parcel profile, LCL, LFC, EL, CAPE, CIN
 - "Indices" – Lifted, K, Total Totals, Showalter
 - "Tropopause" – tropopause pressure and temperature
 - "Wind" – shear and SRH
 - "Misc" – PWAT, freezing level index
 - "CloudBase" – height and type

Usage Example

After retrieving a profile from the database (or reading a NetCDF file), you can compute the diagnostics as follows:

```
import numpy as np
from metpy.units import units
from utils.sounding_diagnostics import full_sounding_diagnostics    # module name suggested for integration

# Example data (normally read from your data source)
pressure = np.array([1000, 925, 850, 700, 500]) * units.hPa
temperature = np.array([20, 15, 10, 0, -20]) * units.degC
relative_humidity = np.array([80, 70, 60, 50, 40]) / 100.0    # dimensionless
u_wind = np.array([5, 10, 15, 20, 25]) * units('m/s')
v_wind = np.array([0, 2, 4, 6, 8]) * units('m/s')
height = np.array([100, 1000, 2000, 3000, 5000]) * units.m

results = full_sounding_diagnostics(
    p=pressure,
    T=temperature,
    RH=relative_humidity,
    u=u_wind,
    v=v_wind,
    z=height
```

```
)

# Access specific results
print(f"CAPE: {results['Parcel']['CAPE']:.0f}")
print(f"CIN: {results['Parcel']['CIN']:.0f}")
print(f"Cloud base: {results['CloudBase']['Height']} ({results['CloudBase']['Type']})")
print(f"Tropopause pressure: {results['Tropopause'][0]:.1f}")
```

The returned quantities are MetPy Quantities, so they retain units and can be converted or formatted as needed.

Integration with the Data Processing Pipeline

While the module is independent, it can be seamlessly integrated into the main processing workflow. For instance, after importing a radiosonde file, you could call `full_sounding_diagnostics` on the retrieved profile and store the derived indices in separate database tables. Users are encouraged to adapt the module to their specific post-processing needs.

9. Automatic Tests

Purpose

Verify that the data processing logic, file reading, and database operations work as expected.

Test Suite Structure

The test suite includes 3 main test modules with 19 total tests:

1. `tests/test_netcdf_reader.py` (8 tests)
 - NetCDF file reading and processing
 - Data validation and transformation
 - Error handling for invalid inputs
2. `tests/test_processor.py` (6 tests)
 - JAR file processing logic
 - Single file processing workflow
 - Database save operations
3. `tests/test_station_manager.py` (5 tests)
 - Station database management
 - Web scraping and data updates
 - Singleton pattern implementation

Running Tests

```
# Run all tests (from project root)
./venv/bin/python -m pytest

# If virtual environment is activated
pytest

# Run with verbose output
pytest -v

# Run specific test module
```

```
pytest tests/test_netcdf_reader.py

# Run with coverage report
pytest --cov=. --cov-report=html
```

Test Output Snapshot

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.4.2, pluggy-1.6.0
cachedir: .pytest_cache
rootdir: /Data/owncloud/src/gruan_retriever
configfile: pytest.ini

tests/test_netcdf_reader.py::TestNetCDFReader::test_process_attribute_value PASSED [ 5%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_process_variable_data_masked PASSED [ 10%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_process_variable_data_small_values PASSED [ 15%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_process_variable_data_strings PASSED [ 21%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_read_netcdf_file_not_found PASSED [ 26%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_read_netcdf_invalid_filename PASSED [ 31%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_read_netcdf_none_zip_ref PASSED [ 36%]
tests/test_netcdf_reader.py::TestNetCDFReader::test_read_netcdf_success PASSED [ 42%]
tests/test_processor.py::TestGRUANProcessor::test_process_single_jar_file_no_files PASSED [ 47%]
tests/test_processor.py::TestGRUANProcessor::test_process_single_jar_file_root_files PASSED [ 52%]
tests/test_processor.py::TestGRUANProcessor::test_process_single_jar_file_with_revisions PASSED [ 57%]
tests/test_processor.py::TestGRUANProcessor::test_process_single_nc_file_skip PASSED [ 63%]
tests/test_processor.py::TestGRUANProcessor::test_process_single_nc_file_success PASSED [ 68%]
tests/test_processor.py::TestGRUANProcessor::test_save_data_copy_method PASSED [ 73%]
tests/test_station_manager.py::TestStationManager::test_load_stations PASSED [ 78%]
tests/test_station_manager.py::TestStationManager::test_scrape_gruan_sites_empty_table PASSED [ 84%]
tests/test_station_manager.py::TestStationManager::test_scrape_gruan_sites_success PASSED [ 89%]
tests/test_station_manager.py::TestStationManager::test_singleton PASSED [ 94%]
tests/test_station_manager.py::TestStationManager::test_update_station_database PASSED [100%]

===== 19 passed in 2.76s =====
```

10. Known Limitations

Current Limitations

1. **Database Dependency**
 - Description: The system is tightly coupled with PostgreSQL.
 - Workaround: None currently.
2. **Memory Usage**
 - Description: Parallel processing of large NetCDF files can consume significant RAM.
 - Workaround: Adjust the number of parallel workers if memory is constrained.

Known Issues

See the issue tracker for a complete list of known bugs.

11. Documentation Summary

Available Documentation

- **README.md** - Quick start and overview.
- **SoftwareUserGuide.md** - This document.

Code Documentation

- Docstrings are provided for all major classes and functions (e.g., `GRUANProcessor`, `StationManager`).
-

12. Support and Contact

Reporting Issues

To report bugs, please contact the development team or open an issue in the repository.

Contributing

Contributions are welcome. Please ensure tests pass before submitting changes.

Maintainer(s):

- Emanuele Tramutola
 - Organization: CNR (National Research Council of Italy)
 - Research Group: C.I.A.O. (CNR-IMAA Atmospheric Observatory)
 - Institute: IMAA (Institute of Methodologies for Environmental Analysis)
 - Location: Tito Scalo (PZ), Italy
-

13. Deliverable Compliance

- ☒ Software placed in public repository
- ☒ Complete source code included
- ☒ Documentation included in repository
- ☒ Installation instructions provided
- ☒ Installation does not require administrator privileges
- ☒ Automatic tests included and verified
- ☒ Snapshot of test output provided
- ☒ License specified
- ☒ Software includes version number/release tag
- ☒ Code follows ECMWF coding standards (if applicable)
- ☒ README.md with quick start instructions
- ☒ Example datasets or data access instructions provided
- ☒ Citation information provided (if applicable)
- ☒ Technical specifications documented